

Servi os e Interfaces para a Cloud - SIC_TSIW

UNIDADE CURRICULAR

Migrating from monolith to microservices

FICHA

Group work: This exercise should be done in groups of 2/3

Deadline: Submit on Moodle until 23h:59 of 13/10/2025

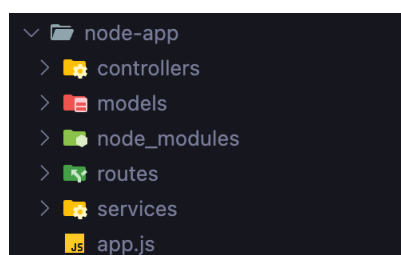
In this exercise, you will analyze a monolithic application and **migrate it into a microservices architecture**. Migrating a legacy monolithic application is a common real-world scenario that often requires significant time and careful planning. This exercise provides a smaller, focused version of this process to help you understand the key concepts and practical steps involved in breaking down a monolith into independent services.

The monolithic application can be found in this repository: <https://github.com/leandrocotsa/sic-monolith-to-microservices-exercise>

Begin by identifying the different business contexts within the monolith and splitting the logic and data into **separate, independent services, with each service managing its own data**. Then, assign each service its own address and introduce an API Gateway to provide a **single unified entry point** for clients, ensuring that requests are redirected to the correct service.

As a result, you should have standalone services that can grow independently, and an API Gateway that redirects the requests to the corresponding service.

Bonus task: Organize the code in each service in a layered architecture, with a clearer division of responsibilities, instead of all the code being inside of *app.js*.



You should submit the **code** of your microservices application and a **diagram** of your architecture.

Suggested steps:

1. Start by having the monolith app running and test its endpoints in the browser
2. Think about which services make sense to be created and what parts of the code would go to which one
3. Create the different services and add the corresponding code to each. Each service is a node app
4. Analyze which services may depend on each other and add the communication between them in the necessary endpoints (you may need to use fetch/axios)
5. Run the services individually and test accessing them in the browser
6. Once they are properly split and working, complete the API gateway configuration file with the missing addresses and try to access each one of the services through the API Gateway's endpoint
7. Try to do the bonus task, each service can have the code better organized inside folders

Technical hints:

- Keep in mind that some services may depend on data from others, so you will also need to consider how inter-service communication is handled. One service may need to call other.
- Regarding the API Gateway, the commands for setting it up can be found in the previous exercise. This time the API gateway will redirect the requests, not load balance them, which is another use case of it. The base configuration file for the API Gateway is present in the repository of this exercise, but it's incomplete.
- You can draw the diagram of the architecture using <https://app.diagrams.net/>